

Итоговое ДЗ, модуль 4 (экзамен). ETL-процессы в Yandex Cloud

Репозиторий с кодом, скриптами и скриншотами: https://github.com/UKhAmster/HSE_ETL_ASONOV

Отчёт собран 14.09.2026 из README.md репозитория. Все ссылки в документе ведут на GitHub.

Студент: Асонов В. **Курс:** ETL-процессы, практическая работа «Реализация ETL-процесса»
Облако: cloud-blizzard-240, каталог default, зона ru-central1-a

PDF-версия отчёта: [Отчёт_Асонов_ETL_модуль4.pdf](#) (собирается из этого README скриптом `scripts/build_report_pdf.py`).

Всё, кроме создания приёмника Object Storage в Data Transfer и дашборда DataLens, выполнено через `yc CLI 1.34.0`, `ydb CLI 2.33.0` и REST API сервисов. Команды сохранены в [scripts/infra_setup.sh](#), скриншоты консоли — в [screenshots/](#).

Структура репозитория:

<code>scripts/</code>	генератор данных, S3-клиент, загрузчик витрин в YDB, обёртка API Airflow, инфраструктура
<code>task1_datatransfer/yql/</code>	YQL: создание таблицы и проверочные запросы
<code>task2_airflow_dataproc/</code>	<code>dags/</code> – DAG для Managed Airflow, <code>jobs/</code> – PySpark-задание
<code>task3_kafka_pyspark/</code>	<code>producer.py</code> – отправка JSON в Kafka, <code>jobs/</code> – PySpark-раскладка JSON
<code>task4_dataLens/</code>	YQL витрин и пошаговая инструкция по дашборду
<code>screenshots/</code>	скриншоты из консоли Yandex Cloud

0. Общая инфраструктура

Ресурс	Имя / ID	Назначение
Сервисный аккаунт	<code>etl-sa (aje51lve2gp8gm5kvdqd)</code>	роли <code>editor</code> , <code>storage.editor</code> , <code>dataproc.agent</code> , <code>dataproc.editor</code> , <code>managed-airflow.integrationProvider</code> , <code>iam.serviceAccounts.user</code> , <code>ydb.editor</code> , <code>vpc.user</code> , <code>kafka.editor</code> , <code>data-transfer.editor</code>
Статический ключ	<code>YCAJEXIFkdXtapziY0R3TKqu2</code>	доступ к Object Storage по S3 API (секрет вне репозитория)
Бакет	<code>hse-etl-asonov-2026</code>	входные данные, PySpark-задания, DAG-файлы, результаты, логи Data Proc
Сеть	<code>default</code> , подсеть <code>default-ru-central1-a (10.128.0.0/24)</code>	все кластеры
NAT-шлюз	<code>etl-nat</code> + таблица маршрутов <code>etl-rt</code>	хосты Data Proc без публичных IP ходят в Object Storage и Maven Central
Группа безопасности	<code>etl-sg (enpu9akb0cnadmce5kta)</code>	внутренний трафик, <code>self-rule</code> (обязательное требование Data Proc), <code>443/9091/8443/22</code> снаружи

Тестовые данные синтетические, генерируются детерминированно скриптом [scripts/generate_data.py](#) (`seed = 42`):

Файл	Строк	Размер	Требование ДЗ
<code>transactions_v2.csv</code>	322 110	32,0 МБ	≥ 30 МБ
<code>applications.csv</code>	456 884	52,0 МБ	≥ 50 МБ

Файл	Строк	Размер	Требование ДЗ
loan_events.jsonl	61 728	22,0 МБ	≥ 20 МБ

Задание 1. Yandex DataTransfer: YDB → Object Storage

1.1. База Managed Service for YDB

```
yc ydb database create etl-db --serverless
# id etn949epluer3vl045t3, endpoint grpcs://ydb.serverless.yandexcloud.net:2135
# database /ru-central1/b1gf5j4p8fki6rqp029/etn949epluer3vl045t3
```

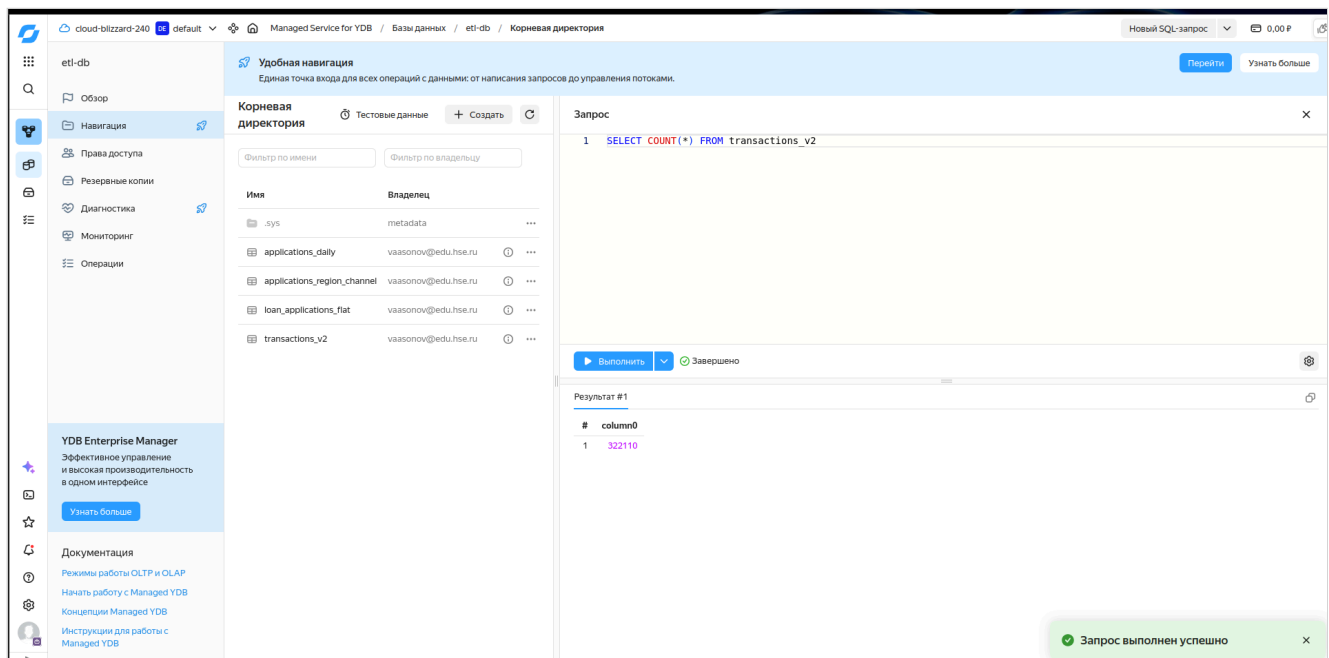
1.2. Таблица и данные

YQL: [task1_datatransfer/yql/01_create_table.sql](#) — таблица `transactions_v2` с первичным ключом `call_id`, `call_time` типа `Datetime`, `duration_sec` `Int32`, `follow_up_required` `Bool`.

Загрузка CSV через `ydb CLI`. Нюанс: колонка `Datetime` при импорте CSV принимает только ISO 8601 (2026-05-01T11:42:15Z), поэтому перед импортом формат даты из ДЗ (2026-05-01 11:42:15) переведён `sed`-ом (см. раздел в [scripts/infra_setup.sh](#)).

```
ydb -e $EP -d $DB yql -f task1_datatransfer/yql/01_create_table.sql
ydb -e $EP -d $DB import file csv --path transactions_v2 --header --newline-delimited data/transactions_v2_iso.csv
# 100% 2.12 MiB/s | 32.3 MiB / 32.3 MiB Elapsed: 15s
```

Проверка ([02_check_data.sql](#)): `COUNT(*) = 322 110` (скриншот консоли: [screenshots/task1_ydb_tables_and_count.png](#)); средняя длительность ответного звонка ≈ 465 с, неотвеченного ≈ 20 с — данные согласованы с генератором.



task1_ydb_tables_and_count.png

1.3. Трансфер

- **Источник** `ydb-transactions-source` (`dtefudhjg35fpfu8lfjq`) создан через REST API `POST https://datatransfer.api.cloud.yandex.net/v1/endpoint C settings.ydbSource (database, instance = ydb.serverless.yandexcloud.net:2135, paths = ["transactions_v2"], аутентификация сервисным`

аккаунтом etl-sa). CLI `yc datatransfer endpoint create` не поддерживает тип YDB, а публичный API не поддерживает приёмник Object Storage, поэтому приёмник создан в консоли.

- **Приёмник** `s3-transactions-target` (`dtetbho13fmloehgrm0e`): тип Object Storage, бакет `hse-etl-asonov-2026`, формат CSV, путь `transfer/transactions_v2`, сервисный аккаунт `etl-sa`.
- **Трансфер** `ydb-to-s3-transactions` (`dtutjs3u94sna1f30lm`), тип «Копирование» (`SNAPSHOT_ONLY`), создан 14.09.2026 в 17:32, активирован из консоли.

1.4. Проверка работоспособности

Статус трансфера — **Завершён** (`yc datatransfer transfer get ... → DONE`). В бакете появился объект

```
transfer/transactions_v2/transactions_v2/part-1789396328-c21f969b.000000.csv  36 453 392 байт (34,8 МБ)
```

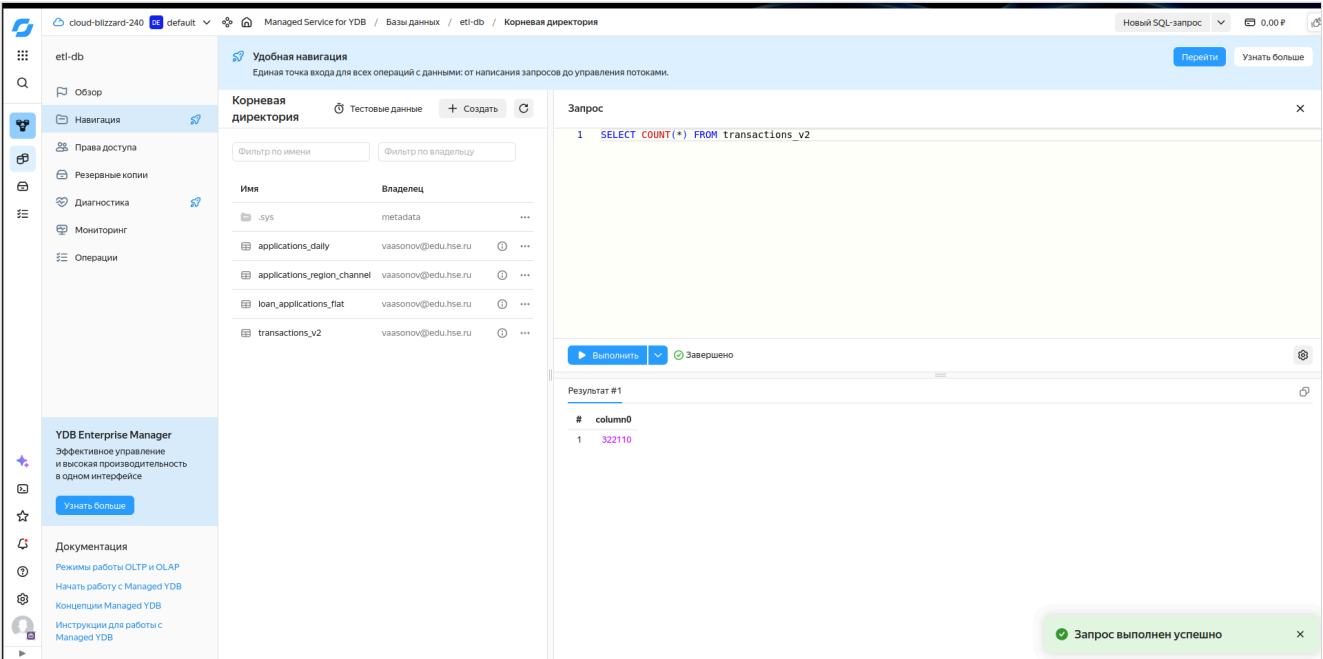
Контроль полноты: в файле **322 110 строк**, столько же, сколько `COUNT(*)` в YDB. Первые строки:

```
call_20260501_0000027,2026-05-01 22:41:31 +0000 UTC,client_41306,DE-NW,credit_card_offer,answered,callback_later,881,true
call_20260501_0000105,2026-05-01 22:34:59 +0000 UTC,client_46870,DE-NW,credit_card_offer,declined,unknown,3,false
```

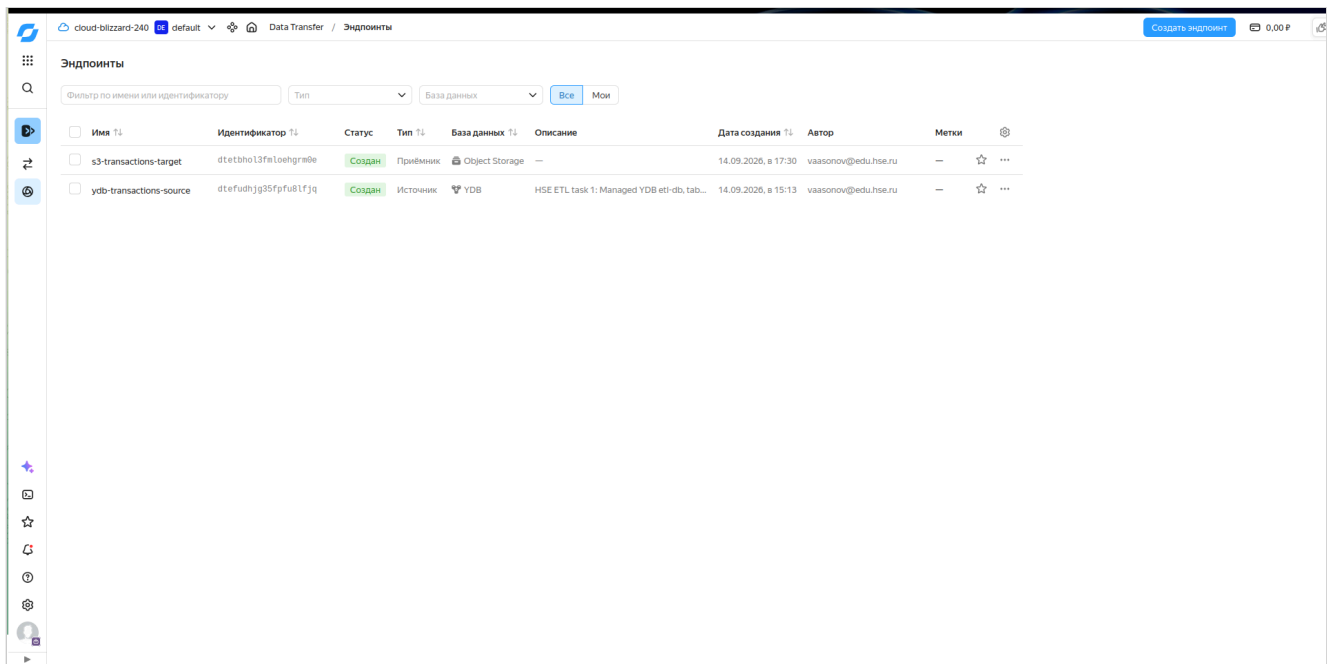
Data Transfer выгружает Datetime в формате `2026-05-01 22:41:31 +0000 UTC`, а Bool — как `true/false`.

Скриншоты:

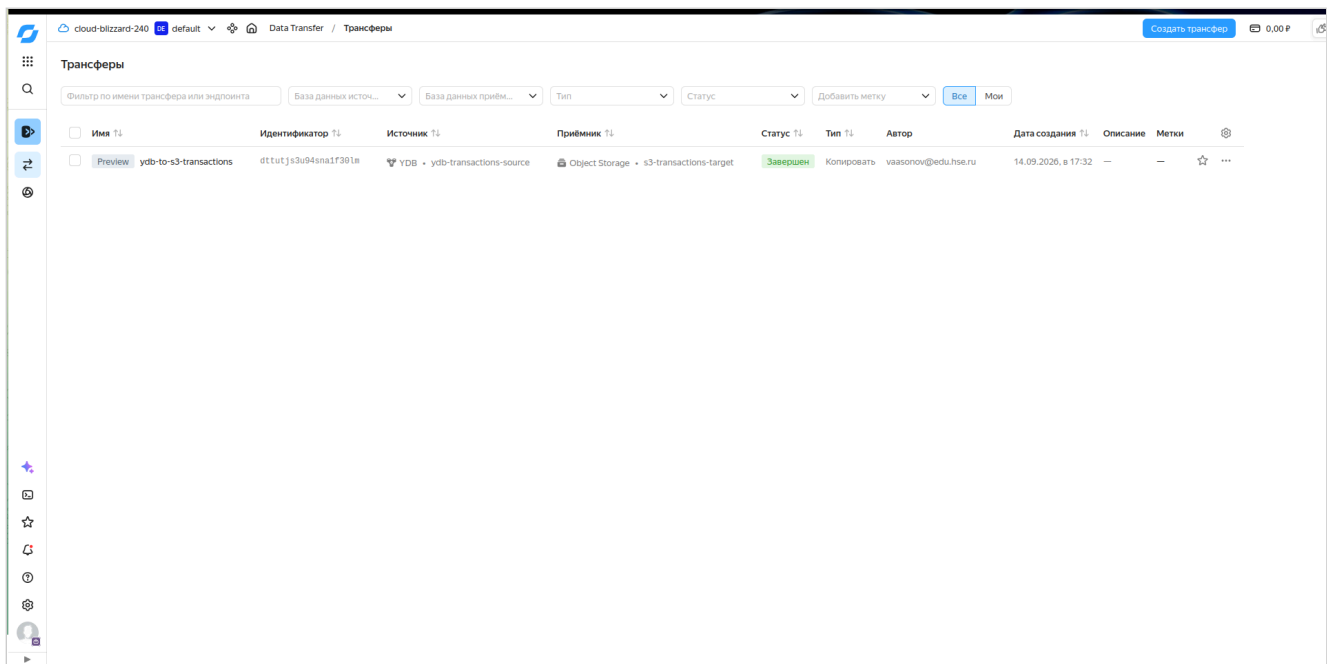
Файл	Что видно
task1_ydb_tables_and_count.png	YDB etl-db: таблицы и <code>SELECT COUNT(*) FROM transactions_v2 = 322 110</code>
task1_dt_endpoints.png	эндпоинты: источник YDB и приёмник Object Storage
task1_dt_transfer_done.png	трансфер <code>ydb-to-s3-transactions</code> , тип «Копировать», статус «Завершён»
task1_bucket_transfer.png	папка <code>transfer/transactions_v2</code> в бакете



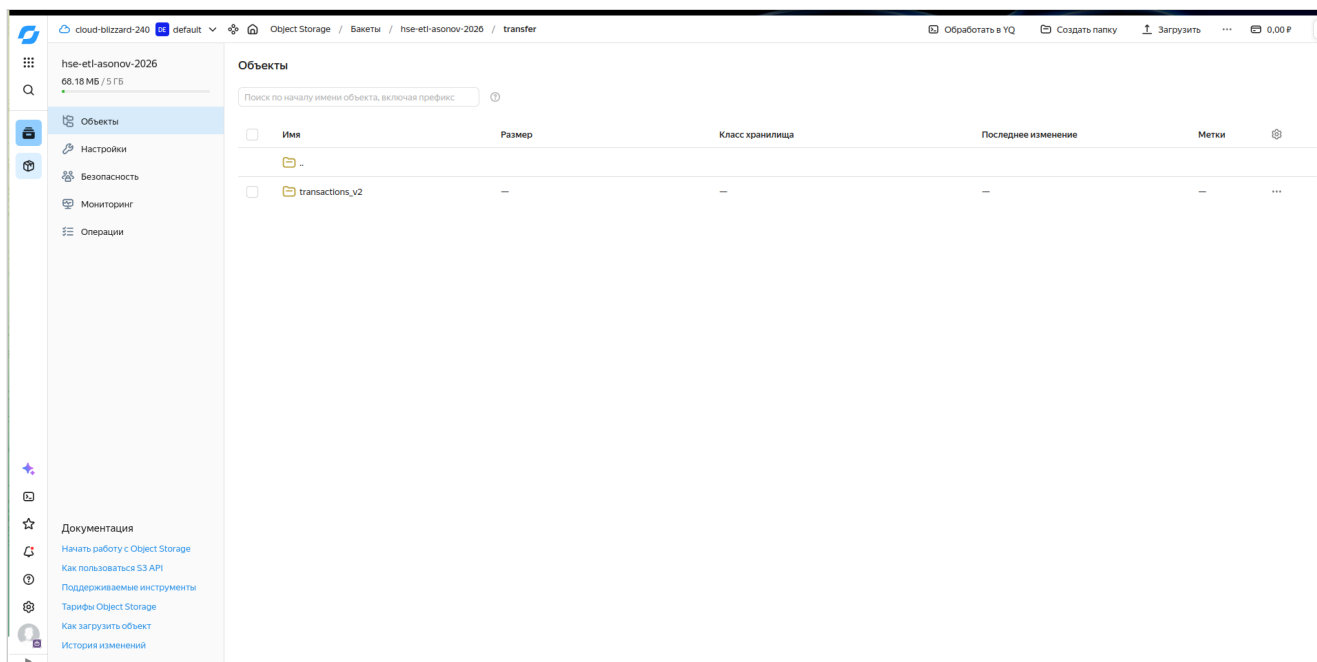
task1_ydb_tables_and_count.png



task1_dt_endpoints.png



task1_dt_transfer_done.png



task1_bucket_transfer.png

Задание 2. Автоматизация Yandex Data Processing через Managed Airflow

2.1. Инфраструктура

- Входной файл `input/applications/2026-05/applications.csv` (54,5 МБ) загружен в бакет скриптом `scripts/s3_upload.py`.
- Кластер **Managed Service for Apache Airflow** `etl-airflow` (Airflow 3.1, провайдер `apache-airflow-providers-yandex 4.3.3`): `webserver`, `scheduler`, `dag-processor`, `worker` — по `c1-m4`; сервисный аккаунт `etl-sa`; DAG-файлы читаются из папки `days/` бакета `hse-etl-asonov-2026`.
- Отдельного подключения с ключом в Airflow не создавалось: провайдер берёт учётные данные сервисного аккаунта кластера из `metadata-сервиса` (в логе: `using metadata service as credentials`).

2.2. PySpark-задание

`task2_airflow_dataproc/jobs/process_applications.py` читает CSV из `s3a://`, приводит `event_time` к `timestamp` и строит три витрины:

- `daily_by_product` — заявки по дню × продукту × решению (количество, суммы, средний скоринг, время обработки);
- `region_channel_conversion` — доля одобрений по регионам и каналам;
- `risk_distribution` — распределение риска по продуктам и число ручных проверок.

Результат — `parquet (coalesce(1))` и CSV-копии в `output/applications/2026-05/`.

2.3. DAG

`task2_airflow_dataproc/dags/dataproc_applications_etl.py`:

```
create_dataproc_cluster -> run_process_applications -> delete_dataproc_cluster (trigger_rule=all_done)
DataprocCreateClusterOperator    DataprocCreatePysparkJobOperator    DataprocDeleteClusterOperator
```

Кластер `airflow-dp-applications`: образ 2.1, сервисы YARN + SPARK, master `s2.small` + 1 compute `s2.small`, диски `network-ssd` 40 ГБ, без датанод (данные в Object Storage). Удаление стоит с `trigger_rule="all_done"`, чтобы кластер не оставался жить после ошибки задания.

Грабли по пути: `cluster_name` у оператора не шаблонизируется, имя `airflow-dp-{{ ds_nodash }}` уходило в API буквально и отклонялось (`Cluster.Name: pattern mismatch`). Заменено на статичное имя.

2.4. Запуск и результат

DAG снят с паузы и запущен через REST API (`scripts/airflow_api.sh`; авторизация двухслойная: `X-Cloud-Authorization: Bearer <IAM>` для прокси Yandex Cloud + JWT самого Airflow из `/auth/token`).

Хронология успешного прогона `manual__2026-09-14T12:35:00` (время UTC):

Задача	Оператор	Старт	Финиш	Итог
<code>create_dataproc_cluster</code>	<code>DataprocCreateClusterOperator</code>	12:35:20	12:38:17	кластер <code>c9q1l3sgdb4642ut5bg</code> создан, <code>id</code> ушёл в <code>XCom</code>
<code>run_process_applications</code>	<code>DataprocCreatePysparkJobOperator</code>	12:38:18	12:39:47	задание <code>c9qmonhtv5llvijlf8cu</code> , статус <code>DONE</code>
<code>delete_dataproc_cluster</code>	<code>DataprocDeleteClusterOperator</code>	12:39:48	~12:42	кластер удалён, <code>yc dataproc cluster list</code> его не показывает

Итог DAG — `success`, весь цикл занял 7 минут.

Скриншоты Airflow и консоли:

Файл	Что видно
<code>task2_airflow_cluster_overview.png</code>	кластер <code>etl-airflow</code> : Airflow 3.1, Python 3.12, SA <code>etl-sa</code> , 4 компонента по <code>c1-m4</code>
<code>task2_bucket_input.png</code> , <code>task2_bucket_jobs.png</code> , <code>task2_bucket_dags.png</code>	бакет: входной CSV, PySpark-задания, DAG-файл
<code>task2_airflow_dags_list.png</code>	список DAG, последний запуск <code>success</code>
<code>task2_airflow_dag_overview_runs.png</code>	три запуска: два отладочных <code>failed</code> , третий <code>success</code> , график длительности
<code>task2_airflow_run_success_tasks.png</code>	успешный запуск: все три задачи <code>Success</code> , операторы и длительности
<code>task2_airflow_audit_log.png</code>	журнал: снятие с паузы и три ручных триггера через REST API
<code>task2_airflow_task_operator.png</code>	задача <code>run_process_applications</code> : <code>DataprocCreatePysparkJobOperator</code> , <code>all_success</code>
(время объектов 12:39:23–12:39:42 совпадает с окном задания):	

cloud-blizzard-240

default

Managed Service for Apache Airflow™

Кластеры

eti-airflow

eti-airflow

Alive

Обзор

Операции

Обслуживание

Логи

Мониторинг

Права доступа

Документация

Создать кластер Apache Airflow

Работа с интерфейсами Apache Airflow

Изменить кластер Apache Airflow

Управление доступом

Тарифы Managed Service for Apache Airflow

История изменений

Обзор

Имяeti-airflow

Идентификаторc9qpcptsmvu9pj24lmsc

Дата создания14.09.2026, в 15:08

Версия Airflow3.1

Версия Python3.12

Сервисный аккаунтeti-sa

Веб-серверПерейти

ДоступностьAlive

Все хосты работают нормально, все запущенные операции были успешно выполнены.

Обслуживание

Обслуживание (UTC)Произвольное время

Сеть

Облачная сетьdefault

Подсетиdefault-ru-central1-a

Зона доступностиru-central1-a

Группы безопасностиeti-sg

Ресурсы

Веб-сервер1

Пресетc1-m4

vCPU1 core

Память4 Гб

Планировщик1

Пресетc1-m4

vCPU1 core

Память4 Гб

ВоркерОт 1 до 1

Пресетc1-m4

vCPU1 core

Память4 Гб

DAG-процессор1

Пресетc1-m4

vCPU1 core

Память4 Гб

19 450,37 P

в месяц

Тарифы и цены

Managed Service for Apache Airflow™. Вычислительные ресурсы кластера, vCPU8 294,40 P

Managed Service for Apache Airflow™. Вычислительные ресурсы кластера, RAM11 155,97 P

task2_airflow_cluster_overview.png

cloud-blizzard-240

default

Object Storage

Бакеты

hse-eti-asonov-2026

Input

hse-eti-asonov-2026

67 MB / 5 GB

Объекты

Настройки

Безопасность

Мониторинг

Операции

Документация

Начать работу с Object Storage

Как пользоваться S3 API

Поддерживаемые инструменты

Тарифы Object Storage

Как загрузить объект

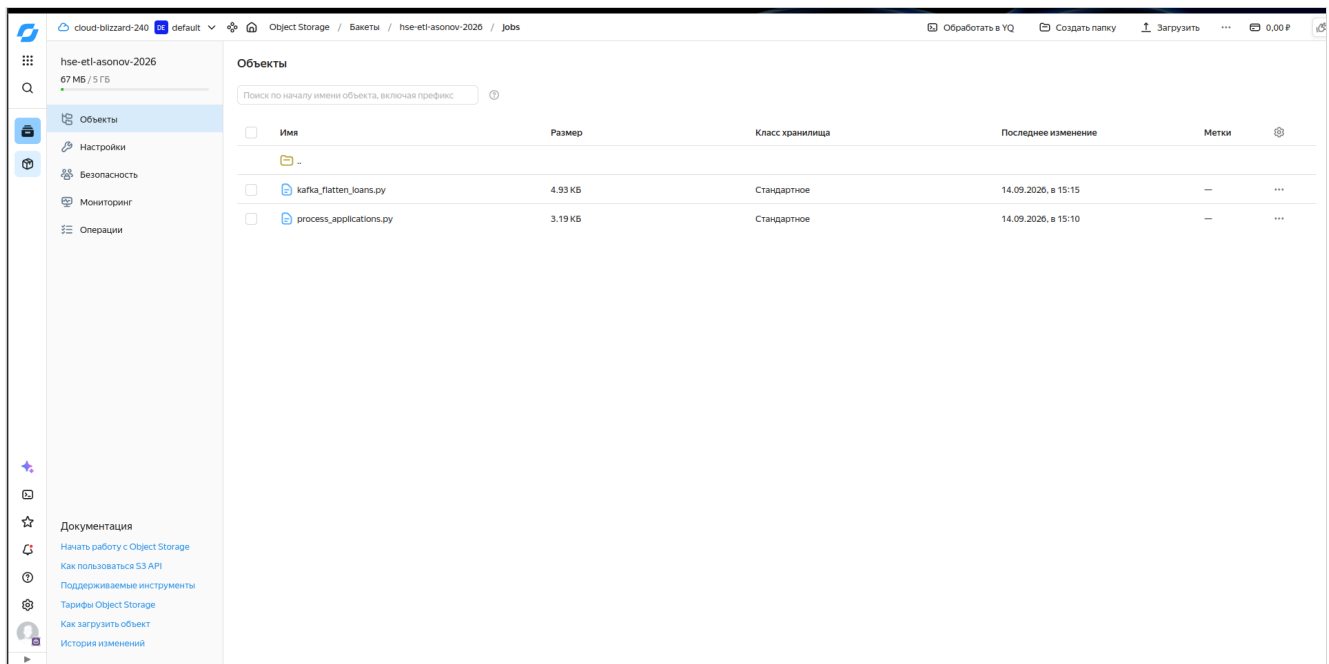
История изменений

Объекты

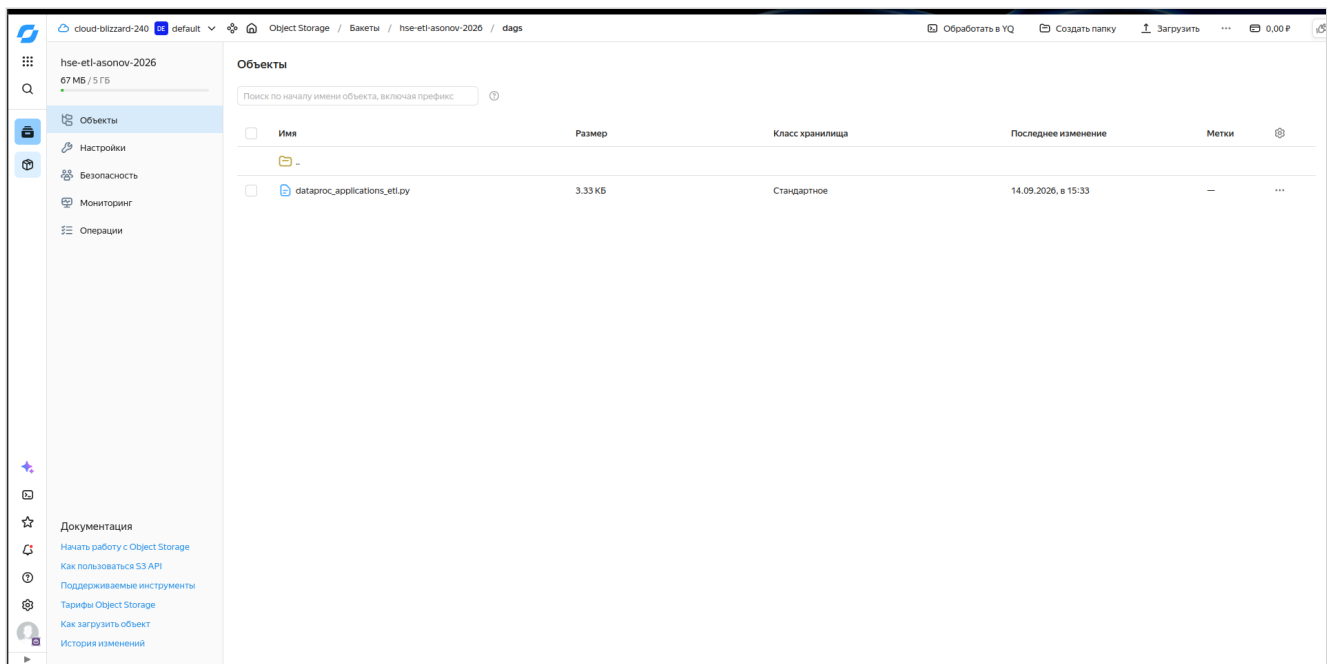
Поиск по началу имени объекта, включая префикс

Имя	Размер	Класс хранилища	Последнее изменение	Метки
-	-	-	-	-
applications	-	-	-	-

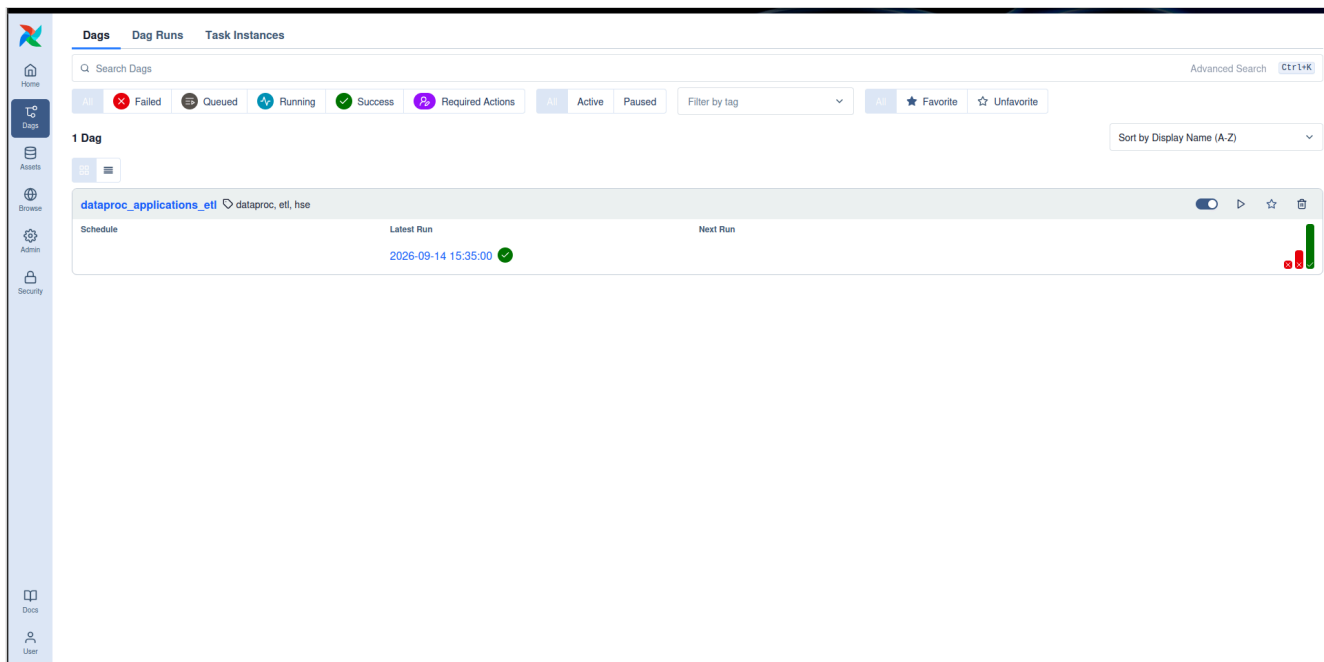
task2_bucket_input.png



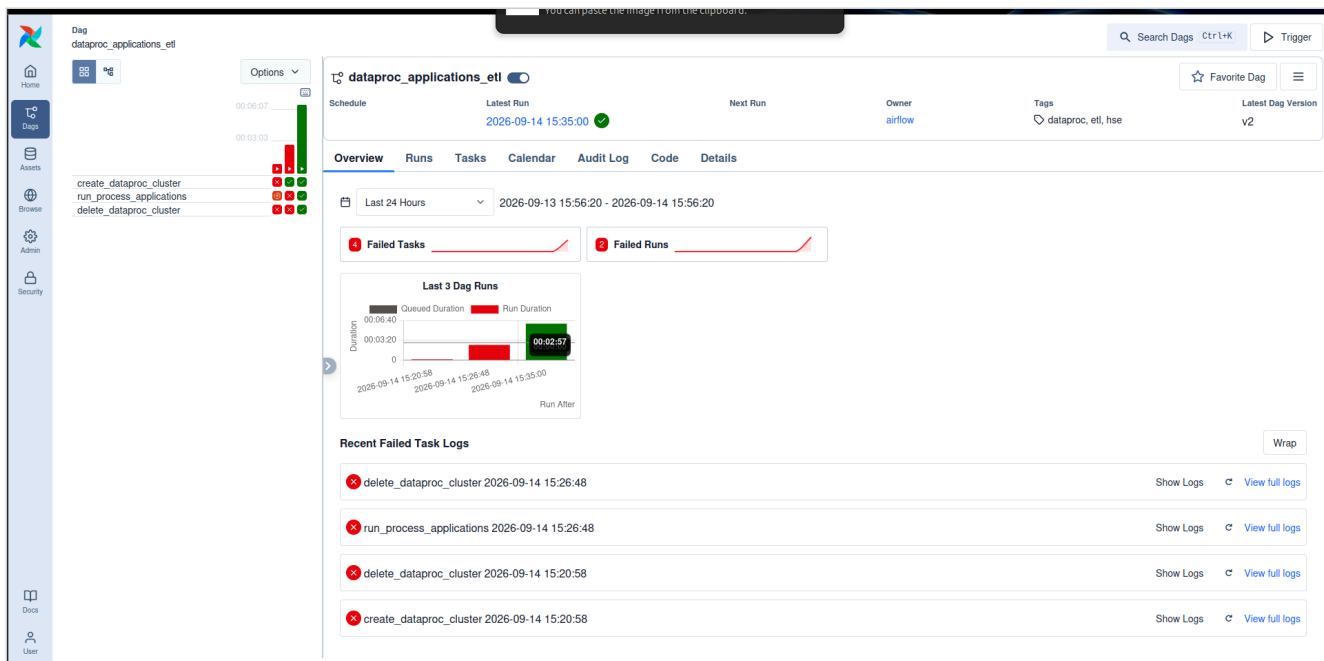
task2_bucket_jobs.png



task2_bucket_dags.png



task2_airflow_dags_list.png



task2_airflow_dag_overview_runs.png

Task Instances screenshot showing successful execution of tasks: delete_dataproc_cluster, run_process_applications, and create_dataproc_cluster.

Task Details:

Task ID	Map Index	State	Start Date	Try Number	Operator	Duration
delete_dataproc_cluster		Success	2026-09-14 15:39:48	1	DataprocDeleteClusterOperator	00:01:18
run_process_applications		Success	2026-09-14 15:38:17	1	DataprocCreatePysparkJobOperator	00:01:30
create_dataproc_cluster		Success	2026-09-14 15:35:01	1	DataprocCreateClusterOperator	00:03:14

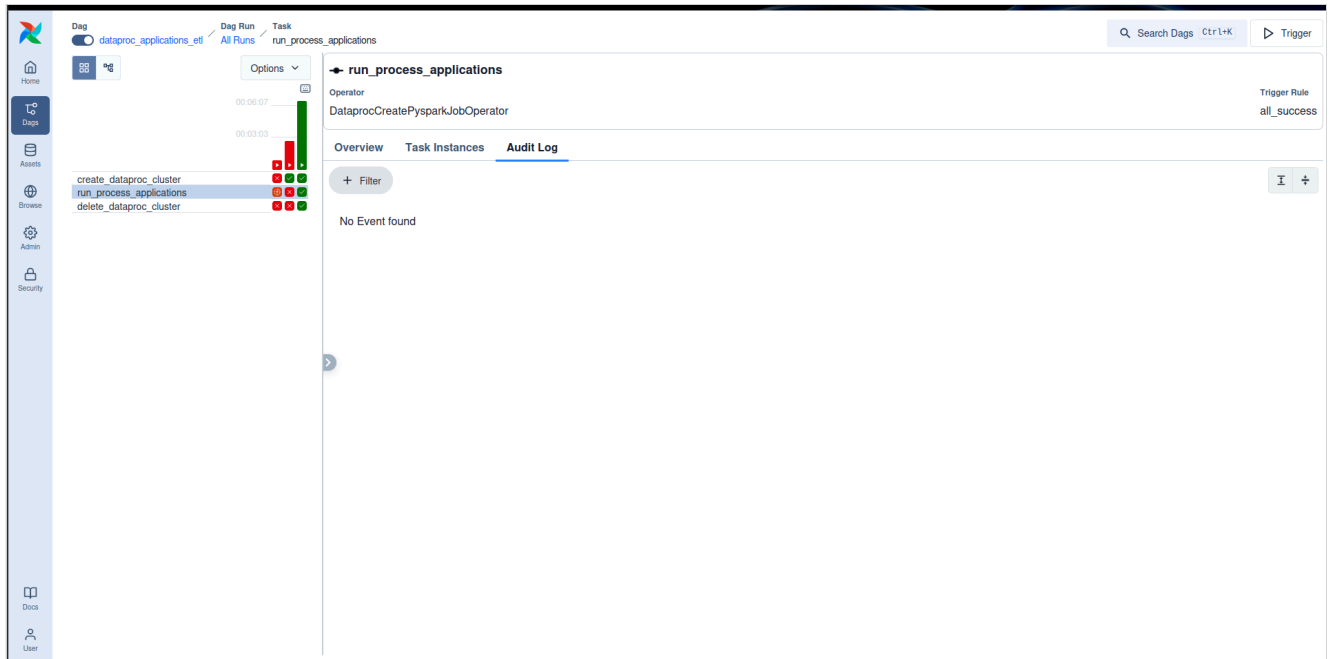
task2_airflow_run_success_tasks.png

Audit Log screenshot showing events for the dagproc_applications_etl DAG.

Audit Log Details:

When	Event	User	Extra	Run ID	Task ID	Map Index	Try Number
2026-09-14 15:35:00	trigger_dag_run	admin	{...} 3 items				
2026-09-14 15:26:48	trigger_dag_run	admin	{...} 3 items				
2026-09-14 15:20:57	trigger_dag_run	admin	{...} 3 items				
2026-09-14 15:20:56	patch_dag	admin	{...} 2 items				

task2_airflow_audit_log.png



task2_airflow_task_operator.png

```
daily_by_product/           parquet 21 КБ   620 строк (день × продукт × решение)
region_channel_conversion/   parquet  3 КБ   40 строк (регион × канал)
risk_distribution/           parquet  2 КБ   15 строк
csv/daily_by_product/, csv/region_channel_conversion/  копии для DataLens
```

Витрины `daily_by_product` и `region_channel_conversion` дополнительно загружены в YDB (`applications_daily`, `applications_region_channel`) для дашборда.

Ещё одни грабли по дороге: провайдер ищет `cluster_id` через `xcom_pull(key="cluster_id")` без `task_ids`, а в Airflow 3 такой вызов читает XCom только своей задачи — обе следующие задачи падали с `Cluster id must be specified`, а кластер оставался жить. Решение: `cluster_id` передаётся явно шаблоном `{{ ti.xcom_pull(task_ids='create_dataproc_cluster', key='cluster_id') }}` (поле templated).

Задание 3. Топики Apache Kafka и PySpark-задание в Data Processing

3.1. Архитектура

```
generate_data.py -> loan_events.jsonl (22 МБ, 61 728 JSON)
| producer.py (kafka-python, SASL_SSL / SCRAM-SHA-512)
v
Managed Kafka etl-kafka (3.9, 1 брокер s3-c2-m8, топик loan_applications, 3 партиции)
| spark.read.format("kafka"), пакет spark-sql-kafka-0-10_2.12:3.0.3
v
Data Proc etl-dp-kafka (2.1, master+compute s2.small) – kafka_flatten_loans.py
| from_json(schema) -> плоские колонки, explode(documents)
v
Object Storage output/loans/{loan_applications_flat, loan_documents, csv/...} -> YDB loan_applications_
flat
```

3.2. Отправка данных

```
yc managed-kafka topic create loan_applications --cluster-name etl-kafka --partitions 3 --replication-factor 1
yc managed-kafka user create etl_user --cluster-name etl-kafka --permission topic=loan_applications,role=producer ...
KAFKA_PASSWORD=... python task3_kafka_pyspark/producer.py data/loan_events.jsonl
# sent 61728 messages, 21.9 MB, 16.1s
```

3.3. PySpark-задание

`task3_kafka_pyspark/jobs/kafka_flatten_loans.py`: batch-чтение топика от `earliest` до `latest`, явная схема `StructType` для JSON, раскладка:

JSON	Колонка
customer.customer_id, customer.region	customer_id, region_code
loan.amount, loan.term_months	loan_amount, term_months
scoring.score, scoring.risk_level	credit_score, risk_level
documents[]	documents_count, documents_verified, document_types (агрегаты) и отдельная таблица <code>loan_documents</code> через <code>explode</code>
submitted_at	submitted_at timestamp + submitted_date

Плюс служебные `partition`, `offset`, `kafka_ts`.

```
yc dataproc job create-pyspark --cluster-name etl-dp-kafka --name kafka_flatten_loans \
--main-python-file-uri s3a://hse-etl-asonov-2026/jobs/kafka_flatten_loans.py \
--args <broker>:9091 --args loan_applications --args etl_user --args '***' --args s3a://hse-etl-asonov-2026/output/loans \
--properties spark.jars.packages=org.apache.spark:spark-sql-kafka-0-10_2.12:3.0.3
```

Результат (лог задания `c9qv90scmthoq0jcannr`, статус `DONE`, ~2 мин):

```
=== events read from kafka: 61728
root
|-- application_id: string
|-- customer_id: string
|-- region_code: string
|-- loan_amount: integer
...
|-- submitted_at: timestamp
```

Скриншоты:

Файл	Что видно
<code>task3_kafka_cluster_overview.png</code>	кластер <code>etl-kafka</code> 3.9.2, <code>s3-c2-m8</code> , 32 ГБ SSD, публичный доступ, SG <code>etl-sg</code>
<code>task3_kafka_topics.png</code>	топик <code>loan_applications</code> , 3 раздела, RF 1
<code>task3_kafka_users.png</code>	пользователь <code>etl_user</code> с ролями <code>producer</code> и <code>consumer</code>
<code>task3_dataproc_cluster_overview.png</code>	кластер <code>etl-dp-kafka</code> 2.1, SPARK + YARN, SA <code>etl-sa</code> , бакет, UI Proxy
<code>task3_dataproc_job_done.png</code>	задание <code>kafka_flatten_loans</code> PYSPARK, статус <code>Done</code> , 15:19-15:21
<code>task3_dataproc_logs_all.png</code> , <code>task3_dataproc_logs_info.png</code>	логи кластера

Файл	Что видно
<code>task3_bucket_output_loans.png</code>	результат в бакете: <code>loan_applications_flat</code> , <code>loan_documents</code> , <code>csv/</code>

cloud-blizzard-240

default

Managed Service for Kafka / Кластеры / etl-kafka

Подключиться

Редактировать

0,00 P

etl-kafka

Alive

Обзор

Топики

Коннекторы

Хосты

Пользователи

Мониторинг

Права доступа

Логи

Операции

Обслуживание

Трансферы

Документация

Как устроен сервис Managed Service for Apache Kafka

Начать работу с Managed Service for Apache Kafka

Подключиться к топикам в кластере Apache Kafka

Топики и разделы

Тарифы Managed Service for Apache Kafka

История изменений

Обзор

Имя: etl-kafka

Идентификатор: c9qoohj7cmtda.jqaedot

Дата создания: 14.09.2026, в 14:55

Окружение: PRODUCTION

Версия: 3.9.2

Регистр схем данных: Нет

Kafka REST API: Нет

Kafka UI: Выключено

Кластер высокодоступен: Нет

Доступность: Alive

Все хосты работают нормально, все запланированные операции были успешно выполнены.

Обслуживание

Обслуживание (UTC): Произвольное время

Ресурсы

КАФКА

Класс хоста: s3-c2-m8 (2 vCPU, 100% vCPU rate, 8 GB RAM)

Хранилище: 32 GB network-ssd

Сеть

Облачная сеть: default

Публичный доступ: Да

Группы безопасности: etl-sg

Дополнительные настройки

Защита от удаления: Выключена

5 856,62 P в месяц

Тарифы и цены

Managed Service for Apache Kafka®. Вычислительные ресурсы (косты: брейкеры Apache Kafka®, Intel Ice Lake, 100% vCPU) 2 505,60 P

Managed Service for Apache Kafka®. Публичный IP-адрес 199,15 P

Managed Service for Apache Kafka®. Вычислительные ресурсы (косты: брейкеры Apache Kafka®, Intel Ice Lake, RAM) 2 649,60 P

Managed Service for Apache Kafka®. Хранилище на сетевых SSD-дисках 502,27 P

task3_kafka_cluster_overview.png

cloud-blizzard-240

default

Managed Service for Kafka / Кластеры / etl-kafka

Создать топик

0,00 P

etl-kafka

Alive

Обзор

Топики

Коннекторы

Хосты

Пользователи

Мониторинг

Права доступа

Логи

Операции

Обслуживание

Трансферы

Документация

Как устроен сервис Managed Service for Apache Kafka

Начать работу с Managed Service for Apache Kafka

Подключиться к топикам в кластере Apache Kafka

Топики и разделы

Тарифы Managed Service for Apache Kafka

История изменений

Топики

Имя

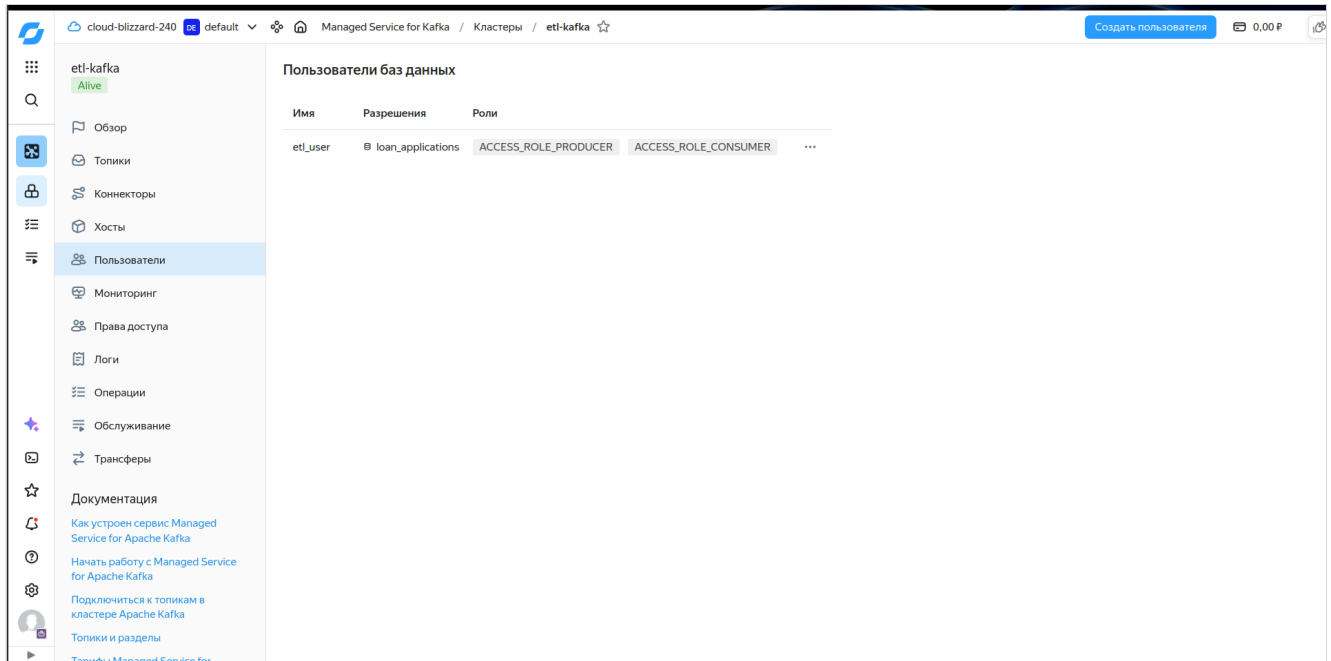
Количество разделов

Фактор репликации

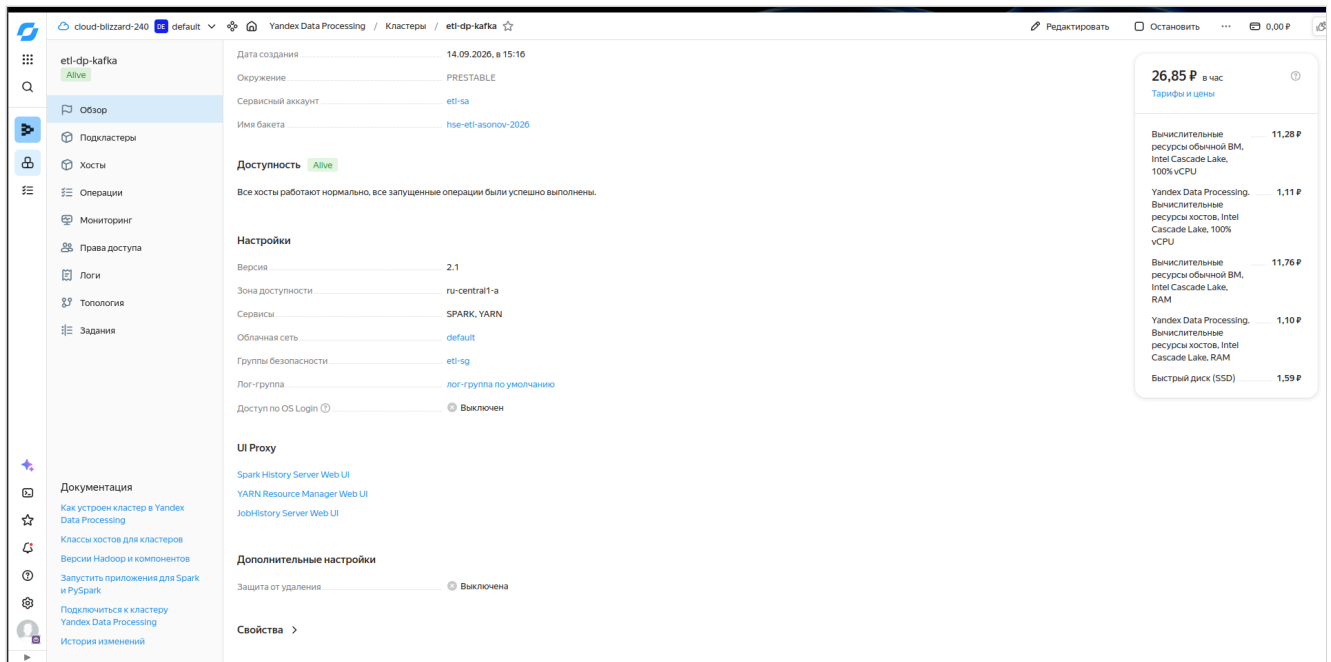
Высокая доступность

loan_applications 3 1 Нет

task3_kafka_topics.png



task3_kafka_users.png



task3_dataproc_cluster_overview.png

cloud-blizzard-240 default Yandex Data Processing / Кластеры / etl-dp-kafka

Создать задание 0,00 P

etl-dp-kafka Alive

Обзор Подкластеры Хосты Операции Мониторинг Права доступа Логи Топология Задания

Документация
Как устроен кластер в Yandex Data Processing
Классы хостов для кластеров
Версии Hadoop и компонентов
Запустить приложения для Spark и PySpark
Подключиться к кластеру Yandex Data Processing
История изменений

Задания

Имя	ID	Статус	Тип задания	Начало	Конец	Пользователь
kafka_flatten_joans	c8y90scmthoqjcanrr	Done	PySPARK	14.09.2026, в 15:19	14.09.2026, в 15:21	vaasonov@edu.hse.ru

task3_dataproc_job_done.png

cloud-blizzard-240 default Yandex Data Processing / Кластеры / etl-dp-kafka

0,00 P

etl-dp-kafka Alive

Обзор Подкластеры Хосты Операции Мониторинг Права доступа Логи Топология Задания

Документация
Как устроен кластер в Yandex Data Processing
Классы хостов для кластеров
Версии Hadoop и компонентов
Запустить приложения для Spark и PySpark
Подключиться к кластеру Yandex Data Processing
История изменений

Логи

Уровень: Все Последний час < > Сейчас 1h 3h 1d 1w 2w 6h

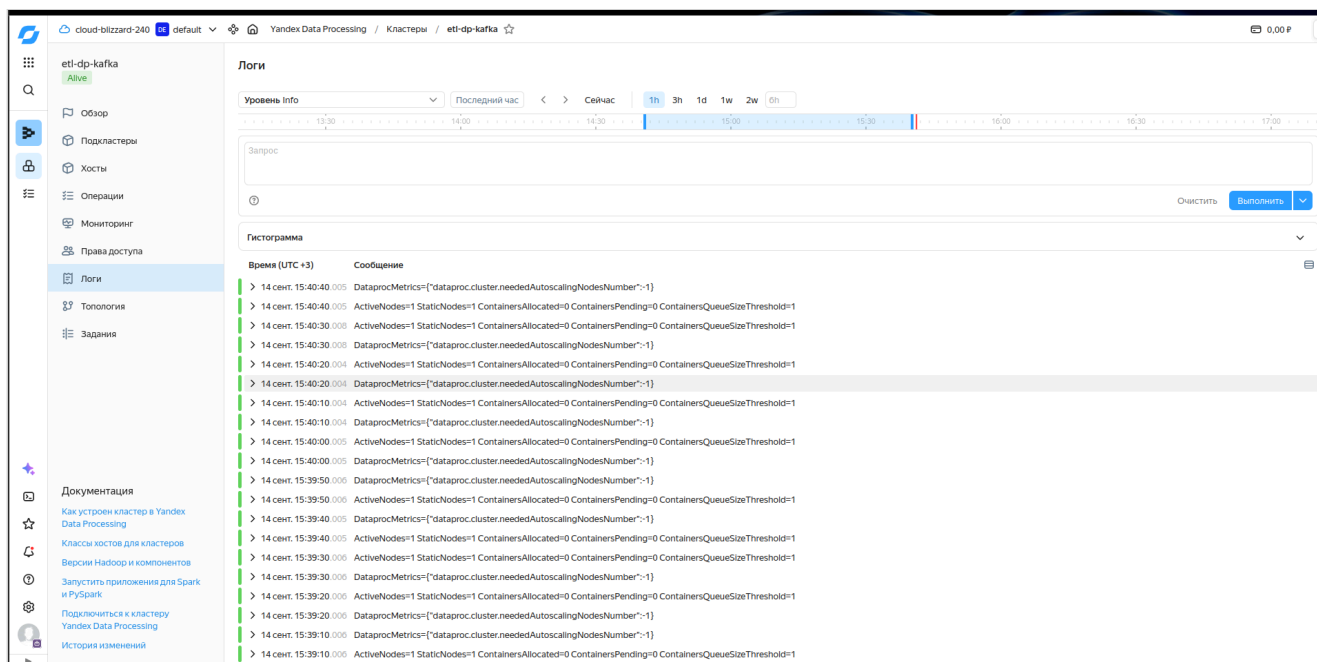
Запрос

Очистить Выполнить

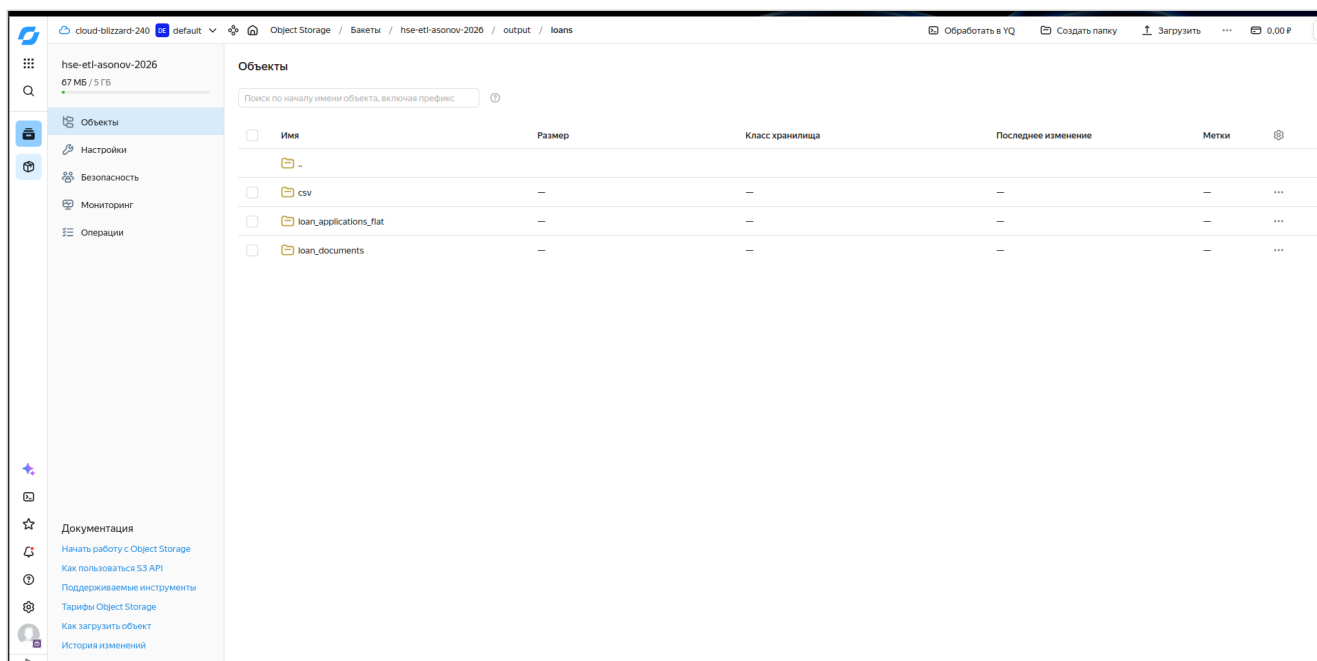
Гистограмма

Время (UTC +3)	Сообщение
14 сент. 15:39:50.000	DataprocMetrics={"dataproc.cluster.neededAutoscalingNodesNumber":-1}
14 сент. 15:39:50.006	ActiveNodes=1 StaticNodes=1 ContainersAllocated=0 ContainersPending=0 ContainersQueueSizeThreshold=1
14 сент. 15:39:50.001	2026-09-14T12:39:50Z [inputs.proctat] Error in plugin: failed to read pidfile "/run/dataproc-agent.pid": open /run/dataproc-agent.pid: no such file or directory
14 сент. 15:39:49.126	Received 0 support jobs from dataproc manager
14 сент. 15:39:49.126	client unary gRPC request finished dataproc-manager.api.cloud.yandex.net:443 /yandex.cloud.dataproc.manager.v1.JobService/ListSupportActive
14 сент. 15:39:49.077	Reply headers: map[content-type:[application/grpc] date:[Mon, 14 Sep 2026 12:39:48 GMT] server:[ycapi] x-server-trace-id:[faf83c41ffff6b597e81554d4e1145f15:35525180f0294814:1]]
14 сент. 15:39:49.077	nothing to decommission
14 сент. 15:39:49.077	new HostsToExclude is empty, so we should skip decommission of the same hosts
14 сент. 15:39:49.077	client unary gRPC request finished dataproc-manager.api.cloud.yandex.net:443 /yandex.cloud.dataproc.manager.v1.DataprocManagerService/Report
14 сент. 15:39:49.041	Request ID: 73a63284-8640-4ff3-aa63-fd2a7a58b020, Trace ID: 450f0d15-1677-4ac7-816b-ea10461f7089
14 сент. 15:39:49.041	client unary gRPC request finished dataproc-manager.api.cloud.yandex.net:443 /yandex.cloud.dataproc.manager.v1.JobService/ListActive
14 сент. 15:39:49.041	client unary gRPC request started dataproc-manager.api.cloud.yandex.net:443 /yandex.cloud.dataproc.manager.v1.JobService/ListSupportActive
14 сент. 15:39:49.041	Received 0 jobs from dataproc manager
14 сент. 15:39:48.994	Request ID: f73ed53c-24f2-45a4-906b-ae85b08f74f5, Trace ID: 789c783a-ecf1-496e-8f35-59244f1fb4ff
14 сент. 15:39:48.994	client unary gRPC request started dataproc-manager.api.cloud.yandex.net:443 /yandex.cloud.dataproc.manager.v1.DataprocManagerService/Report
14 сент. 15:39:48.994	YARN Info: [AvailableLiveNodes:[{"Name":"1a-dataproc-c-5te5ik9f24td00m.mdb.yandexcloud.net State:RUNNING NumContainers:0 UsedMemoryMB:0 AvailableMemoryMB:12288 UpdateTime:1789389496237"}]]
14 сент. 15:39:48.992	client unary gRPC request started dataproc-manager.api.cloud.yandex.net:443 /yandex.cloud.dataproc.manager.v1.JobService/ListActive
14 сент. 15:39:48.992	Request ID: eda27907-cb64-4030-6d8a-77d974dbdfc3, Trace ID: df201667-1855-40ce-b773-aa93353a8716
14 сент. 15:39:44.140	Received 0 support jobs from dataproc manager
14 сент. 15:39:44.140	client unary gRPC request finished dataproc-manager.api.cloud.yandex.net:443 /yandex.cloud.dataproc.manager.v1.JobService/ListSupportActive

task3_dataproc_logs_all.png



task3_dataproc_logs_info.png



task3_bucket_output_loans.png

В бакете: `loan_applications_flat` parquet 1,66 МБ + CSV 9,7 МБ, `loan_documents` parquet 0,6 МБ, `csv/loan_summary`. Плоская таблица дополнительно загружена в YDB (`loan_applications_flat`, 61 728 строк) скриптом `scripts/load_results_to_ydb.py` для DataLens.

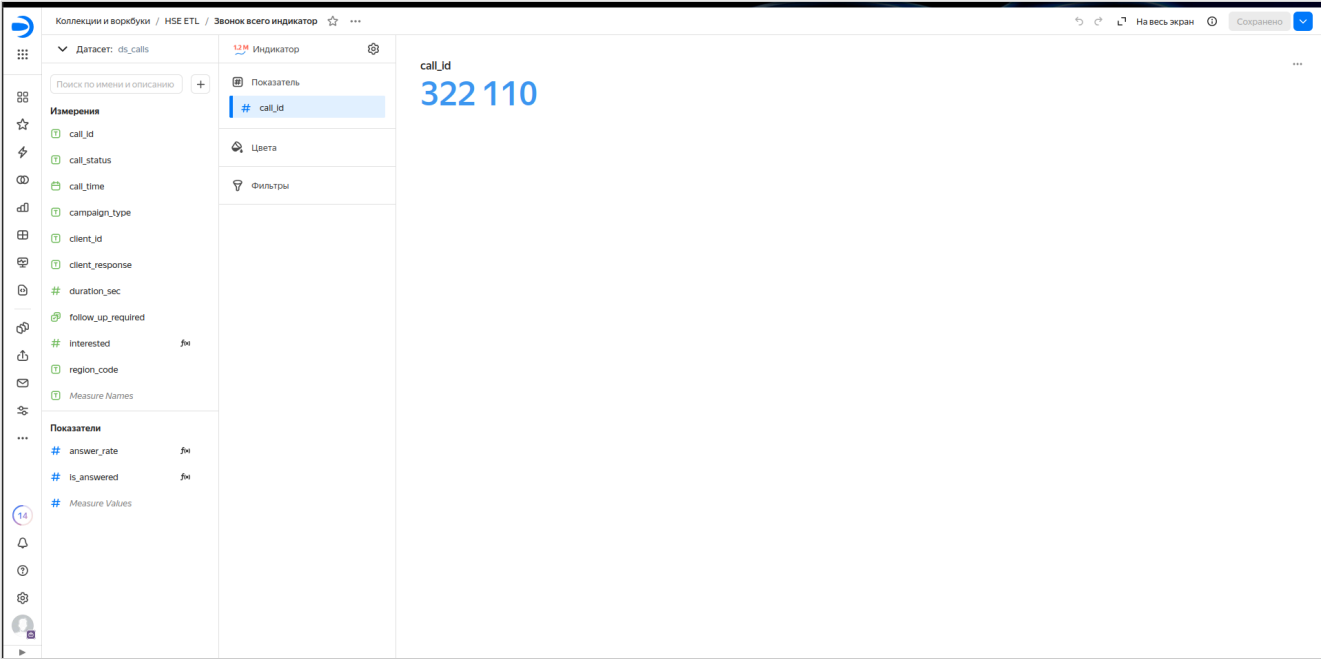
Задание 4. Визуализация в DataLens

Пошаговая инструкция и состав дашборда: [task4_dataLens/README.md](#). Источник — YDB `etl-db`, подключение `ydb-etl-db` через сервисный аккаунт `etl-sa`, воркбук HSE ETL.

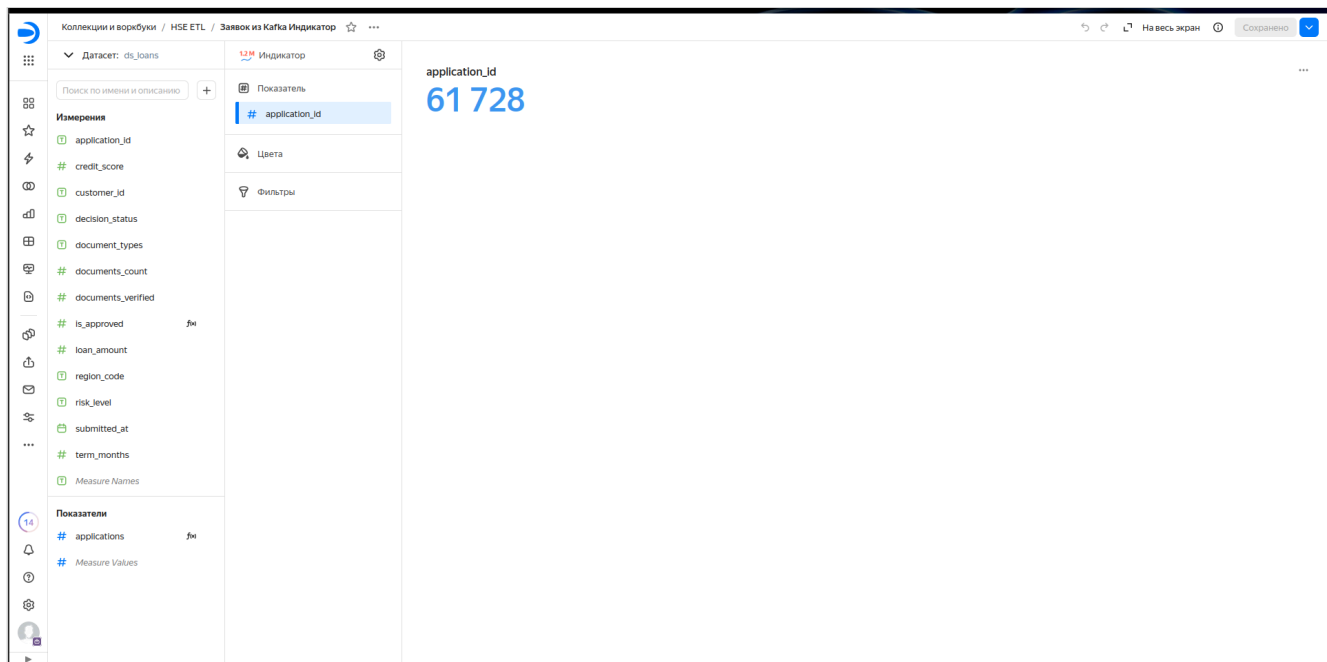
Датасеты (по одному на таблицу): `ds_calls` (`transactions_v2`), `ds_loans` (`loan_applications_flat`), `ds_region_channel` (`applications_region_channel`), `ds_daily` (`applications_daily`). Вычисляемые поля построены на флагах `IF(..., 1, 0)` с агрегацией в чарте: `COUNTIF` для YDB-источника DataLens не поддерживает.

Чарты (Wizard) и что на них видно:

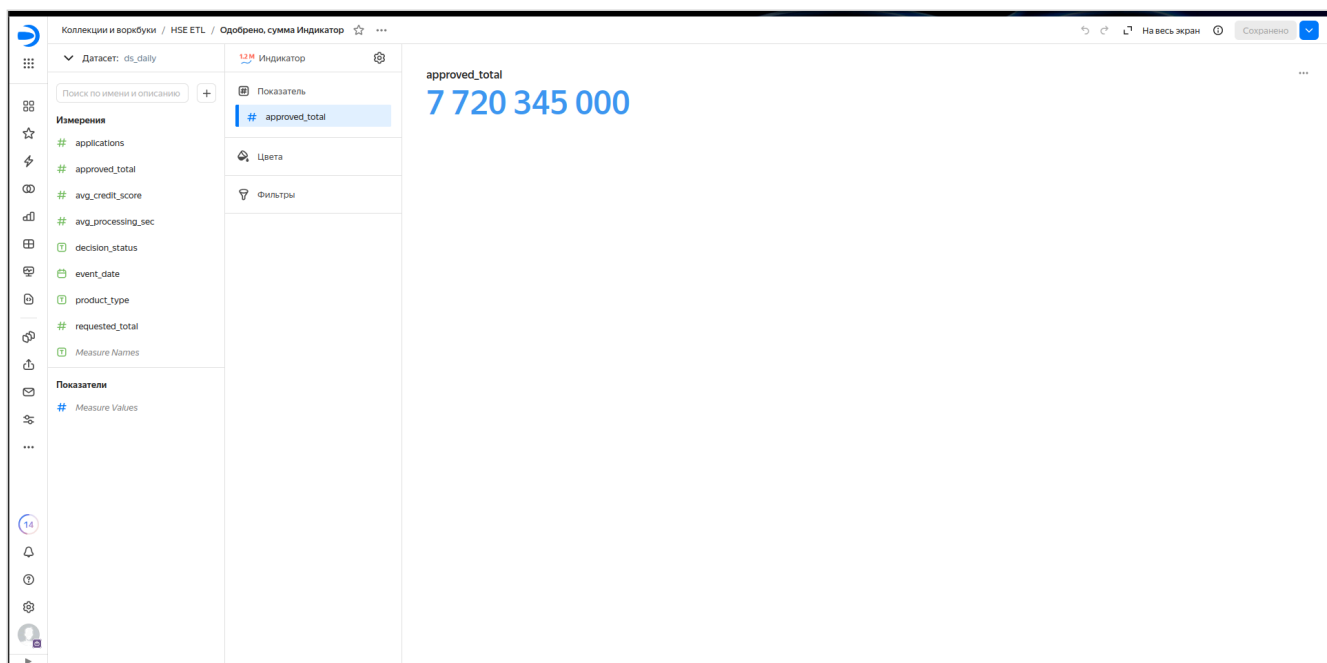
Файл	Чарт	Датасет	Наблюдение
task4_indicator_calls_total.png	Звонков всего	ds_calls	322 110 — совпадает с COUNT(*) в YDB
task4_indicator_kafka_applications.png	Заявок из Kafka	ds_loans	61 728 — совпадает с числом сообщений продюсера
task4_indicator_approved_total.png	Одобрено, сумма	ds_daily	7 720 345 000
task4_chart_calls_by_campaign.png	Звонки по кампаниям и статусам	ds_calls	~64 тыс. звонков на кампанию, статусы распределены равномерно
task4_chart_answer_rate_by_region.png	Доля дозвонков по регионам	ds_calls	~0,20 во всех регионах, лидер DE-NW
task4_chart_duration_by_day.png	Длительность разговора по дням	ds_calls	фильтр call_status = answered, средняя ~465 с
task4_chart_risk_x_decision_pivot.png	Риск x решение (сводная с раскраской)	ds_loans	high/approved 17 216, цифры совпадают с YQL-проверкой
task4_chart_loan_amount_by_day.png	Сумма кредитов по дням и риску	ds_loans	стек high/low/medium по дням мая
task4_chart_region_channel_table.png	Одобрение по регионам и каналам	ds_region_channel	approval_rate 0,54–0,56, ~11,4 тыс. заявок на пару
task4_chart_applications_by_product.png	Заявки по продуктам (Airflow-витрина)	ds_daily	~14,7 тыс. заявок в день, 5 продуктов поровну



task4_indicator_calls_total.png



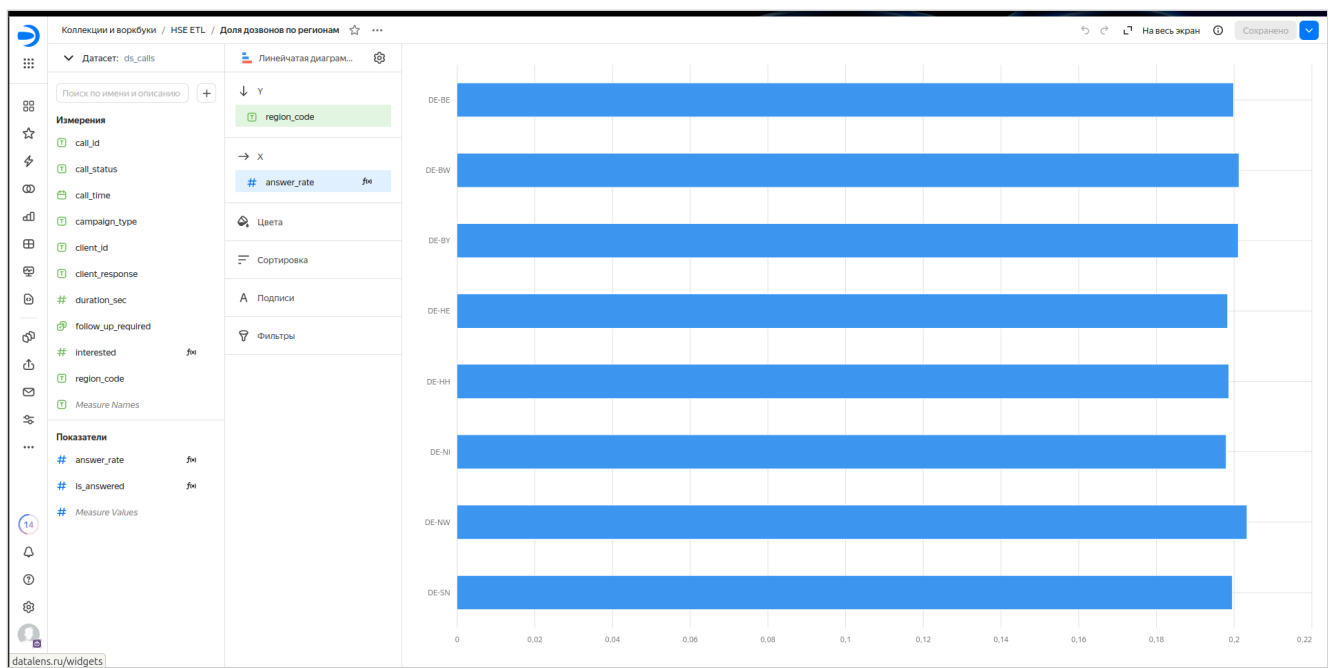
task4_indicator_kafka_applications.png



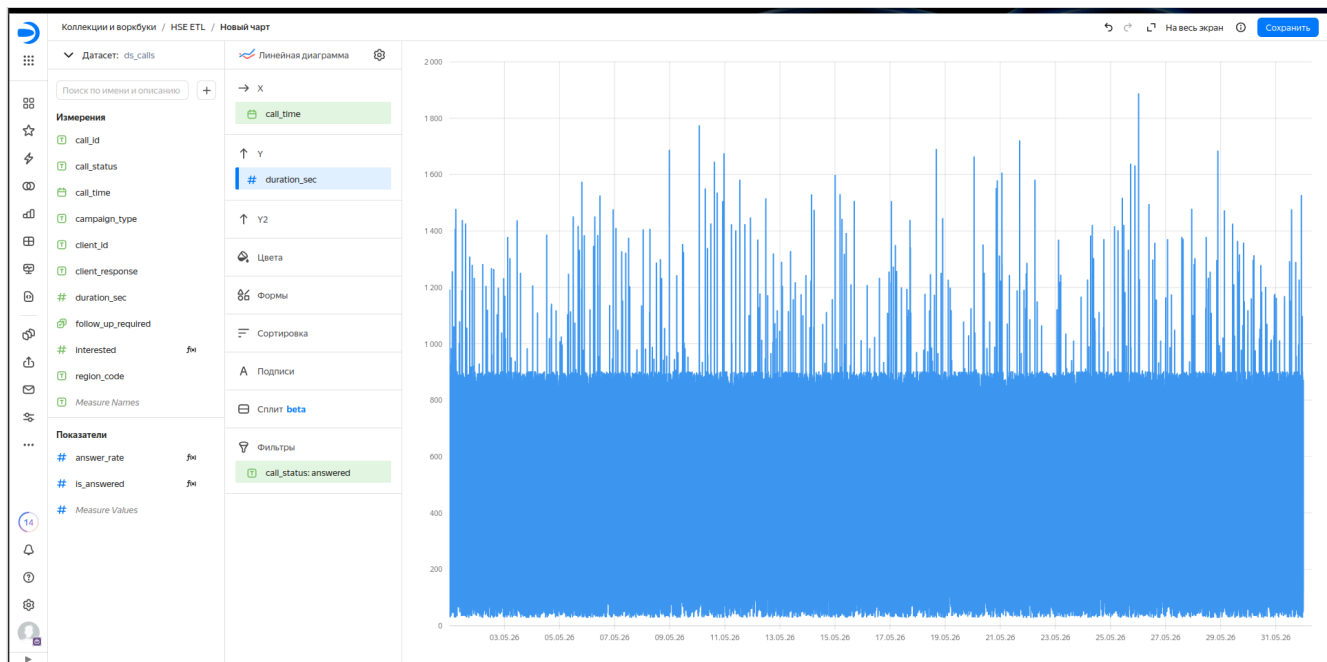
task4_indicator_approved_total.png



task4_chart_calls_by_campaign.png



task4_chart_answer_rate_by_region.png



Коллекции и воркбуки / HSE ETL / Заявки из Kafka

Датасет: ds_loans

Поиск по имени и описанию

Измерения

- application_id
- credit_score
- customer_id
- decision_status
- document_types
- documents_count
- documents_verified
- is_approved
- loan_amount
- region_code
- risk_level
- submitted_at
- term_months
- Measure Names

Показатели

- applications
- Measure Values

decision_status

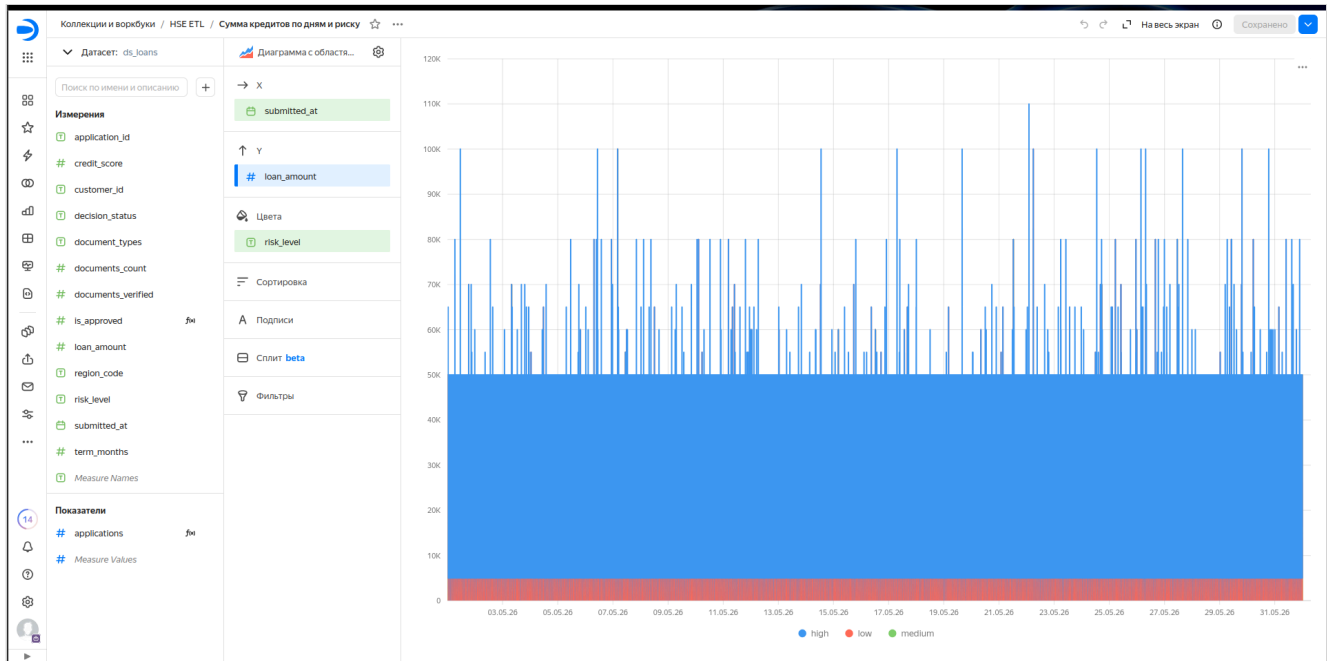
risk_level

application_id

applications

risk_level	approved	manual_review	pending	rejected
high	17 216	4 730	1 053	7 096
low	9 366	2 530	854	4 270
medium	7 321	2 013	699	3 366

task4_chart_risk_x_decision_pivot.png



task4_chart_loan_amount_by_day.png

Коллекции и воркбуки / HSE ETL / Одобрение по регионам и каналам

Датасет: ds_region_channel

Поиск по имени и описанию

Измерения

- applications
- approval_rate
- approved
- avg_requested
- channel
- region_code

Таблица

Столбцы

- approval_rate
- applications
- channel
- region_code

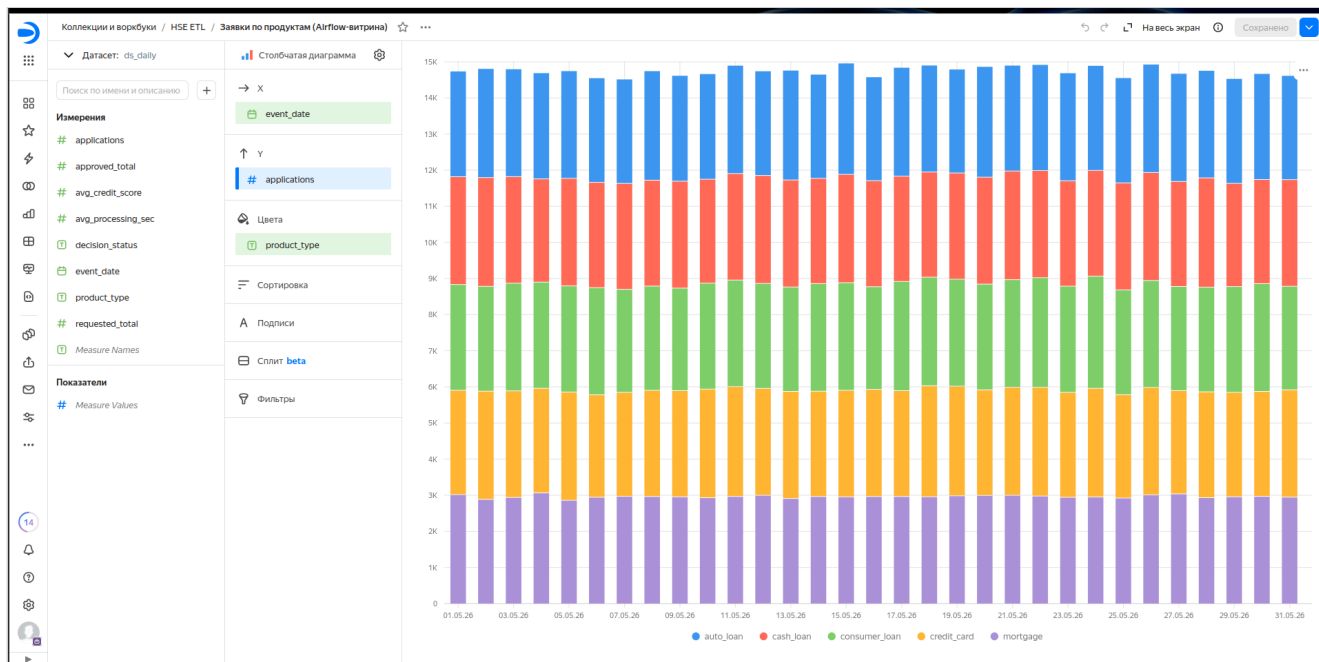
Цвета

Сортировка

Фильтры

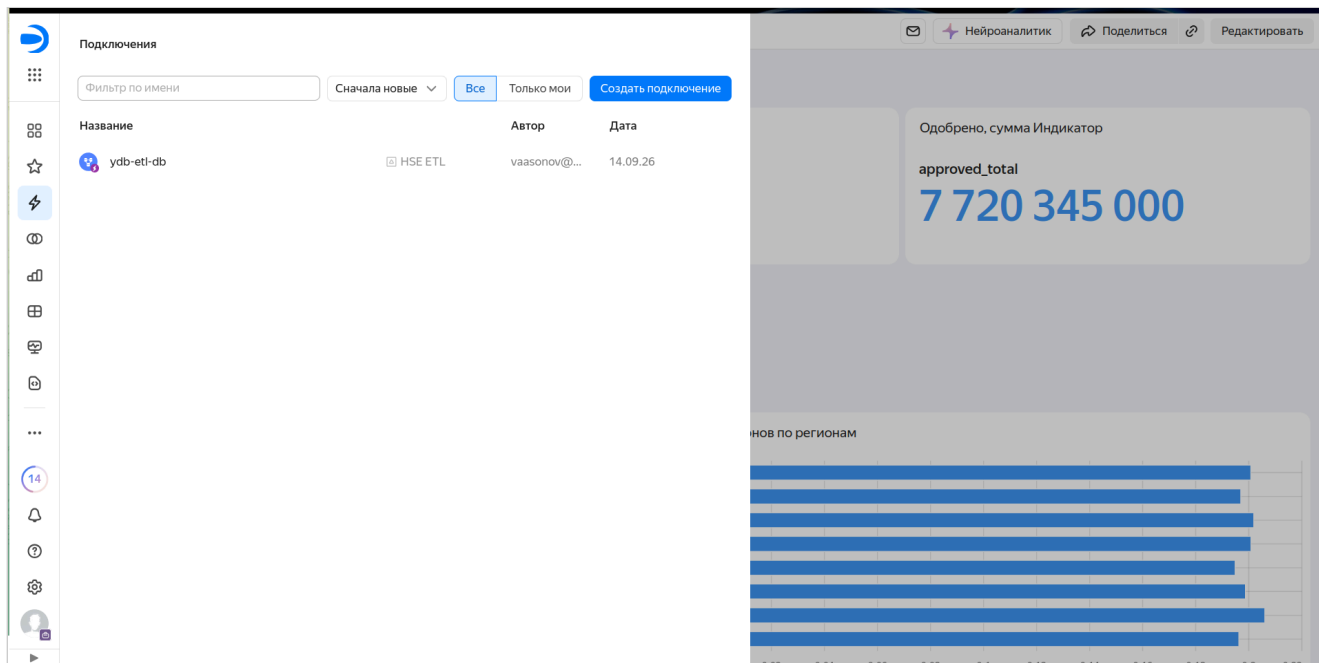
approval_rate	applications	channel	region_code
0.54	11 403	web	DE-BY
0.54	11 542	partner	DE-SN
0.54	11 294	partner	DE-NI
0.54	11 448	partner	DE-BE
0.54	11 346	web	DE-NW
0.54	11 432	call_center	DE-NI
0.55	11 309	call_center	DE-BE
0.55	11 297	branch	DE-BE
0.55	11 450	web	DE-NI
0.55	11 486	mobile	DE-BW
0.55	11 626	partner	DE-BW
0.55	11 368	web	DE-HE
0.55	11 413	branch	DE-SN
0.55	11 280	branch	DE-HH
0.55	11 514	partner	DE-HH
0.55	11 227	mobile	DE-HE
0.55	11 515	branch	DE-BW
0.55	11 310	call_center	DE-HE
0.55	11 515	call_center	DE-NW
0.55	11 437	partner	DE-HE
0.55	11 368	mobile	DE-BY
0.55	11 441	partner	DE-BY
0.55	11 500	mobile	DE-BE
0.55	11 203	call_center	DE-BW
0.55	11 351	call_center	DE-HH
0.55	11 374	branch	DE-NW
0.55	11 456	call_center	DE-BY
0.55	11 405	mobile	DE-NW

task4_chart_region_channel_table.png

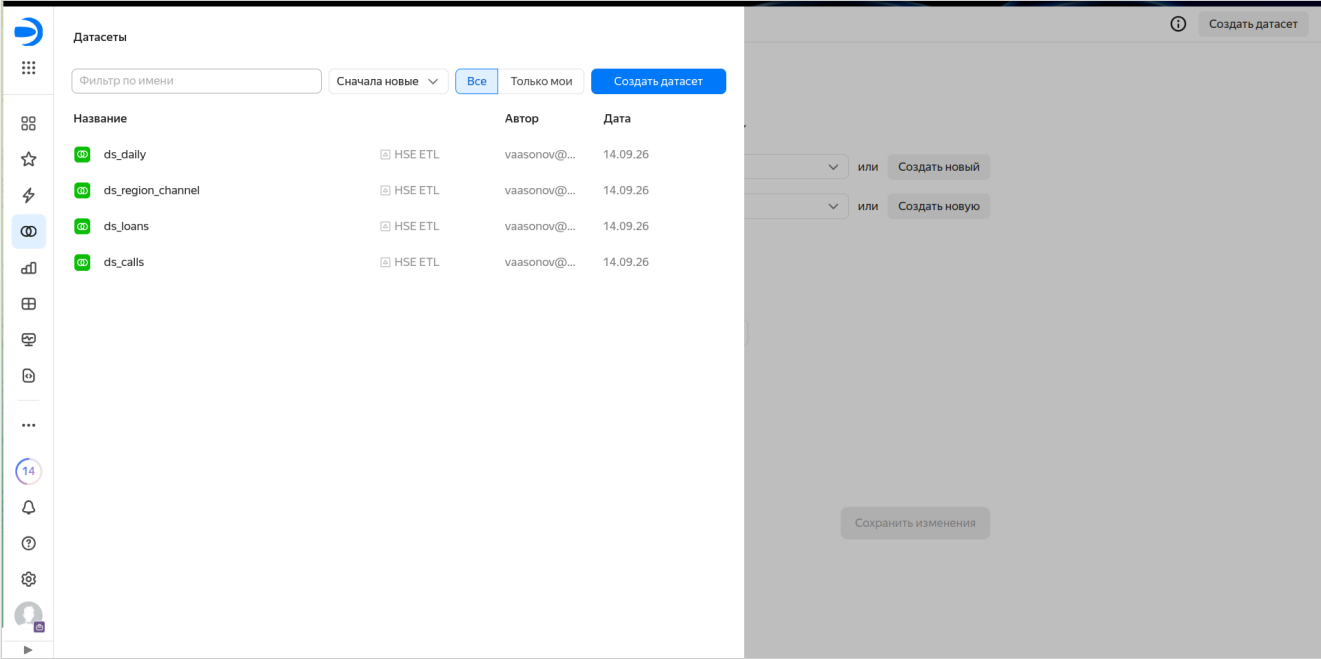


task4_chart_applications_by_product.png

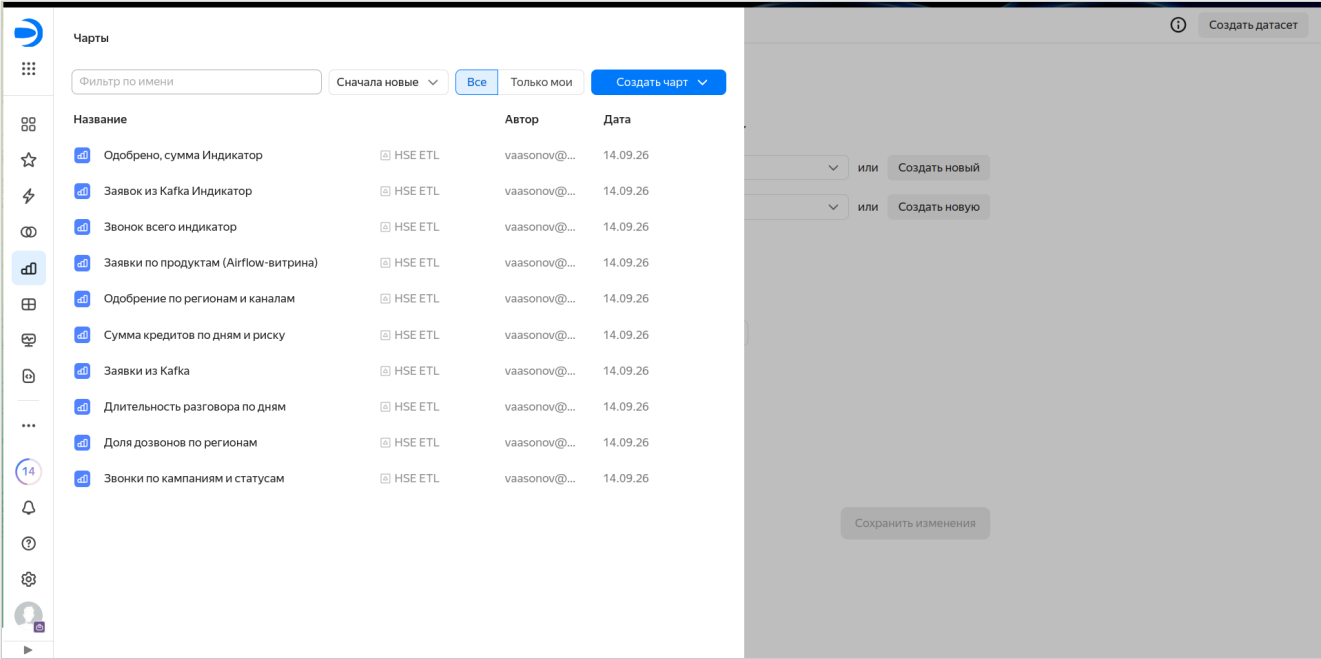
Объекты воркбука HSE ETL: [task4_dataLens_connection.png](#) (подключение ydb-etl-db), [task4_dataLens_datasets.png](#) (4 датасета), [task4_dataLens_charts.png](#) (10 чартов).



task4_dataLens_connection.png



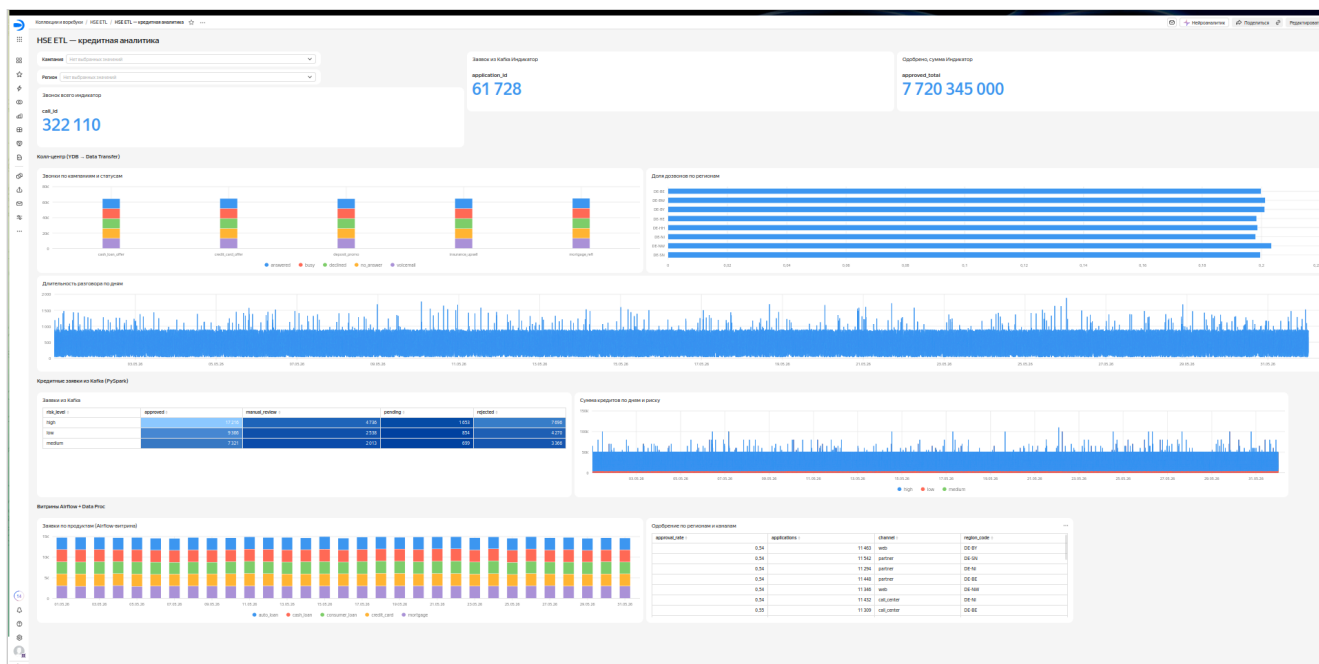
task4_datalens_datasets.png



task4_datalens_charts.png

Дашборд HSE ETL – кредитная аналитика: селекторы Кампания (ds_calls → 4 виджета) и Регион (ds_loans → 3 виджета), три секции с заголовками.

Файл	Что видно
task4_dashboard_full.png	дашборд без фильтров: 322 110 звонков, 61 728 заявок, 7,72 млрд одобрено
task4_dashboard_filtered.png	выбраны кампания cash_loan_offer и регион DE-BE: индикаторы пересчитались (64 234 звонка, 7 608 заявок), витрины Airflow не зависят от селекторов



task4_dashboard_full.png

